

并行与分布式计算

-- 架构

架构风格 Architectural styles

基本思想 Basic idea

一种风格（**style**）的表述体现在以下几个方面：

- ▶ 具有明确定义接口的（可替换）**组件**；
- ▶ 组件之间的**连接**方式；
- ▶ 组件之间交换的**数据**；
- ▶ 这些组件和连接器如何共同配置成一个系统。

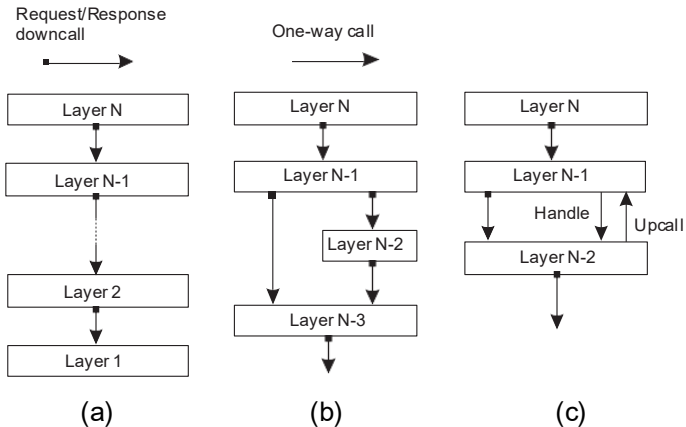
连接器 Connector

一种用于中介组件间的通信、协调或协作的机制。

例如：（远程）过程调用、消息传递或数据流等设施。

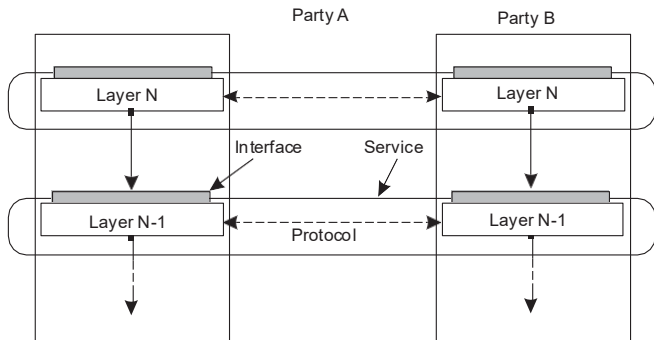
分层架构Layered architecture

不同分层组织方式



示例: 通信协议 communication protocols

协议 Protocol, 服务 service, 借口 interface



双方通信示例Two-party communication

Server

```
1 from socket import *
2 s = socket(AF_INET, SOCK_STREAM)
3 (conn, addr) = s.accept() # returns new socket and addr. client
4 while True:               # forever
5     data = conn.recv(1024) # receive data from client
6     if not data: break     # stop if client stopped
7     conn.send(str(data)+"*") # return sent data plus an "*"
8 conn.close()              # close the connection
```

Client

```
1 from socket import *
2 s = socket(AF_INET, SOCK_STREAM)
3 s.connect((HOST, PORT)) # connect to server (block until accepted)
4 s.send('Hello, world') # send some data
5 data = s.recv(1024)    # receive the response
6 print data             # print the result
7 s.close()              # close the connection
```

应用分层Application Layering

传统三层视图

- 应用接口层包含用于与用户或外部应用程序交互的单元
- 处理层包含应用程序的功能，即不涉及特定数据
- 数据层包含客户希望通过应用程序组件进行操作的数据

应用分层Application Layering

传统三层视图

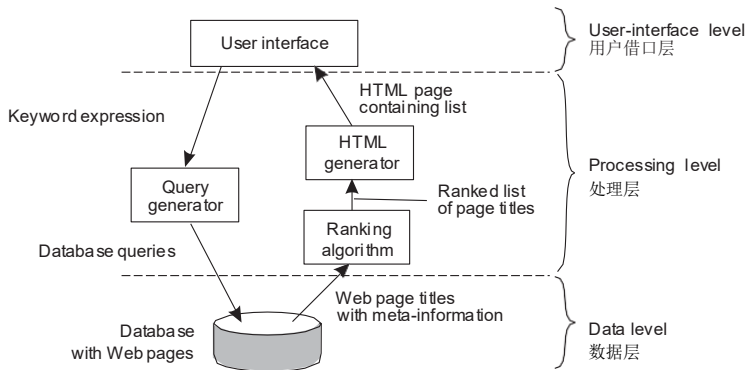
- 应用接口层包含用于与用户或外部应用程序交互的单元
- 处理层包含应用程序的功能，即不涉及特定数据
- 数据层包含客户希望通过应用程序组件进行操作的数据

观察

这种分层常见于许多采用传统数据库技术及配套应用的分布式信息系统中。

应用分层Application Layering

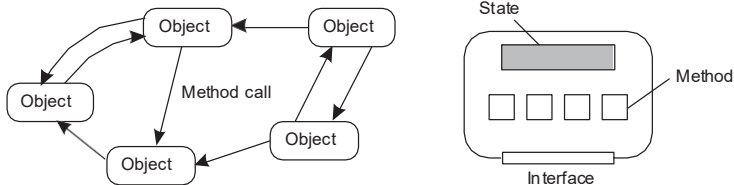
示例: 搜索引擎



基于对象的设计风格Object-based style

本质Essence

组件即对象，通过过程调用相互连接。对象可部署在不同机器上，因此调用可以跨网络执行。



封装Encapsulation

对象**封装数据**，并提供**操作该数据**的方法，而不暴露其内部实现。

RESTful 架构

本质 Essence

将分布式系统视为一组资源，每个资源由组件独立管理。
资源可被（远程）应用程序添加、移除、检索和修改。

1. 资源通过统一的命名方案进行标识
2. 所有服务提供相同的接口
3. 发送至服务或从服务发出的消息均完全自描述
4. 服务在执行完操作后，会忘记所有关于调用者的信息

基本操作 Basic operations

Operation	Description
PUT	Create a new resource
GET	Retrieve the state of a resource in some representation
DELETE	Delete a resource
POST	Modify a resource by transferring a new state

示例: Amazon 存储服务

Essence

对象（即文件）被放置于**存储桶**（即目录）中。存储桶不能嵌套存储桶。对存储桶 **BucketName** 中的对象 **ObjectName** 进行操作需要以下标识符：

<http://BucketName.s3.amazonaws.com/ObjectName>

典型操作

所有操作均通过发送 HTTP 请求执行：

- 创建存储桶/对象：使用 **PUT** 方法，附带 **URI**
- 列举对象：对存储桶名称执行 **GET** 请求
- 读取对象：对完整 **URI** 执行 **GET** 请求

On interfaces

问题

许多人喜欢RESTful方法，因为服务接口非常简洁。但代价是大量工作需要在参数空间中完成。

Amazon S3 SOAP interface

Bucket operations	Object operations
ListAllMyBuckets	PutObjectInline
CreateBucket	PutObject
DeleteBucket	CopyObject
ListBucket	GetObject
GetBucketAccessControlPolicy	GetObjectExtended
SetBucketAccessControlPolicy	DeleteObject
GetBucketLoggingStatus	GetObjectAccessControlPolicy
SetBucketLoggingStatus	SetObjectAccessControlPolicy

关于接口

简化

假设有一个存储桶接口，它提供了一个创建操作，需要输入一个字符串（例如 **mybucket**），用于创建名为“mybucket”的存储桶。

关于接口

简化

假设有一个存储桶接口，它提供了一个创建操作，需要输入一个字符串（例如 **mybucket**），用于创建名为“mybucket”的存储桶。

SOAP

```
import bucket  
bucket.create("mybucket")
```

关于接口

简化

假设有一个存储桶接口，它提供了一个创建操作，需要输入一个字符串（例如 **mybucket**），用于创建名为“mybucket”的存储桶。

SOAP

```
import bucket  
bucket.create("mybucket")
```

RESTful

```
PUT "http://mybucket.s3.amazonaws.com/"
```

关于接口

简化

假设有一个存储桶接口，它提供了一个创建操作，需要输入一个字符串（例如 **mybucket**），用于创建名为“mybucket”的存储桶。

SOAP

```
import bucket  
bucket.create("mybucket")
```

RESTful

```
PUT "http://mybucket.s3.amazonaws.com/"
```

结论

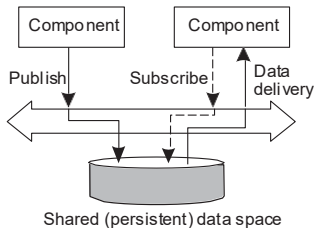
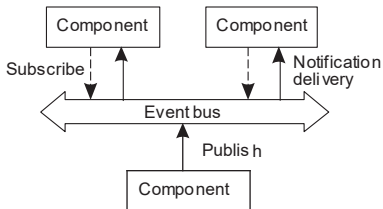
Are there any to draw?

协调

时间耦合与引用耦合Temporal and referential coupling

	Temporally coupled	Temporally decoupled
Referentially coupled	Direct	Mailbox
Referentially decoupled	Event-based	Shared data space

基于事件的架构与共享数据空间



利用遗留系统构建中间件

Using legacy to build middleware

问题

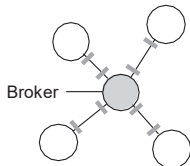
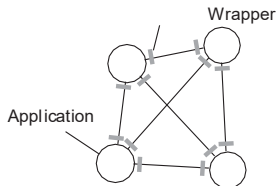
遗留组件所提供的接口很可能无法满足所有应用的需求。

解决方案

包装器（wrapper）或**适配器（asapter）**为客户端应用提供一个可接受的接口。它将客户端的请求功能转换为遗留组件所能提供的功能。

组织包装器Organizing wrappers

两种解决方案：一对一或通过代理



N个应用的复杂度:

- ▶ 一对一: requires $N \times (N - 1) = O(N^2)$ wrappers
- ▶ 代理: requires $2N = O(N)$ wrappers

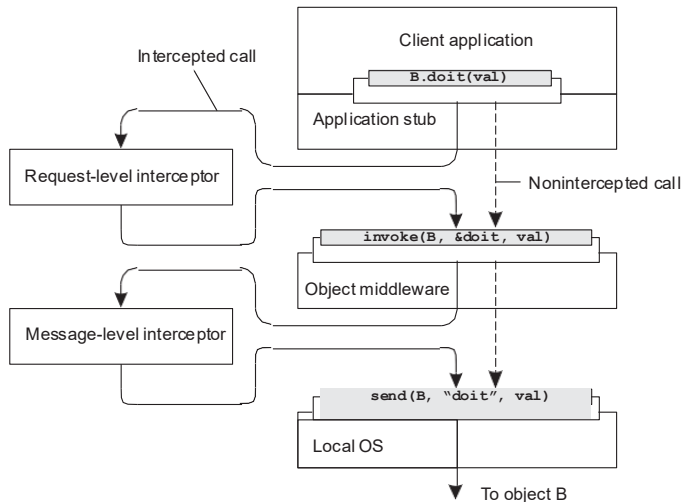
开发可适应的中间件

问题

中间件提供了适用于**大多数**应用的解决方案
⇒ 你可能需要针对特定应用调整其行为。

Intercept the usual flow of control

拦截常规控制流

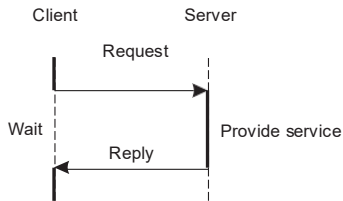


集中式系统架构 Centralized system architectures

基本客户端-服务器模型

特征:

- 存在提供服务的过程（服务器）
- 存在使用服务的过程（客户端）
- 客户端和服务端可以位于不同的机器上



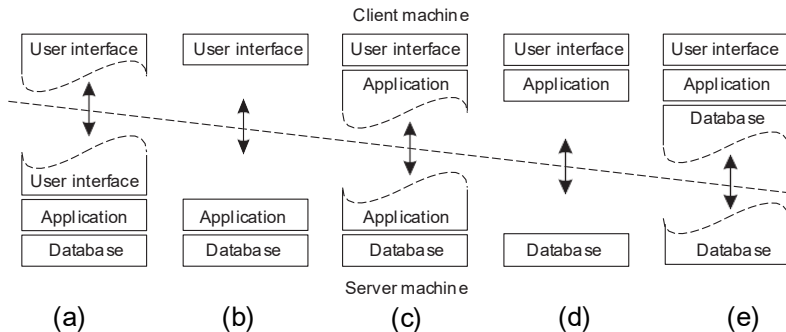
多层集中式系统架构

Multi-tiered centralized system architectures

一些组织形式

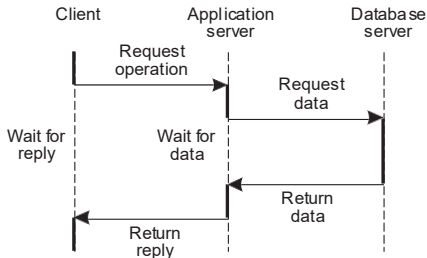
- 单层：哑终端/大型机配置
- 双层：客户端/单服务器配置
- 三层：每一层位于独立的机器上

传统的双层配置



同时作为客户端和服务端

三层架构



替代性组织形式

垂直分布

源于将分布式应用划分为三个逻辑层，并将每一层的组件运行在不同的服务器（机器）上。

水平分布

客户端或服务器可以在物理上拆分为逻辑上等价的多个部分，但每个部分只操作完整数据集中属于自己的那部分数据。

对等架构

所有进程都是平等的：每个进程都承担需要执行的功能 $\Rightarrow$ 每个进程将同时充当客户端和服务器（即充当**服务者**）。

结构化 P2P

Essence 本质

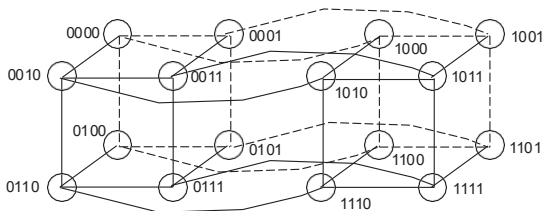
利用**无语义索引**: 每个数据项唯一关联一个键, 该键被用作索引。

常见做法: 使用哈希函数

$$\text{key}(\text{data item}) = \text{hash}(\text{data item's value}).$$

P2P系统现在负责存储 (key, value) 对

示例: hypercube



Looking up d with **key** $k \in \{0, 1, 2, \dots, 2^4 - 1\}$ means **routing** request to node with **identifier** k .

非结构化P2P

本质

每个节点维护一个自组织的邻居列表。由此形成的覆盖网络类似于**随机图**：边 (u, v) 仅以特定概率 $P[(u, v)]$ 存在。

搜索

- ▶ **泛洪**：发起节点 u 将请求 d 传递给所有邻居。当接收节点之前已见过该请求时，则忽略该请求。否则，节点 v 在本地搜索 d （递归进行）。可通过**生存时间**（TTL，即最大跳数）加以限制。
- ▶ **随机漫步**：发起节点 u 将请求 d 传递给随机选择的邻居 v 。如果 v 没有 d ，则将其请求转发给自己随机选择的一个邻居，以此类推。

洪泛 versus 随机游走

模型

假设有 N 个节点，每个数据项被复制到随机选择的 r 个节点上。

随机游走

$P[k]$ 经过 k 次尝试后找到该数据项的概率

$$P[k] = \frac{r}{N} \left(1 - \frac{r}{N}\right)^{k-1}.$$

S (“搜索规模”) 是预期需要探测的节点数:

$$S = \sum_{k=1}^N k \cdot P[k] = \sum_{k=1}^N k \cdot \frac{r}{N} \left(1 - \frac{r}{N}\right)^{k-1} \approx N/r \text{ for } 1 \ll r \leq N.$$

洪泛 versus 随机游走

洪泛

- ▶ 向随机选择的 d 个邻居进行泛洪
- ▶ 经过 k 步后，大约会有 $R(k) = d \cdot (d - 1)^{(k-1)}$ 个节点被触及（假设 k 较小）。
- ▶ 当拥有数据的节点比例为 r/N 时，如果 $(r/N) \cdot R(k) \geq 1$ ，我们将找到该数据项。

比较

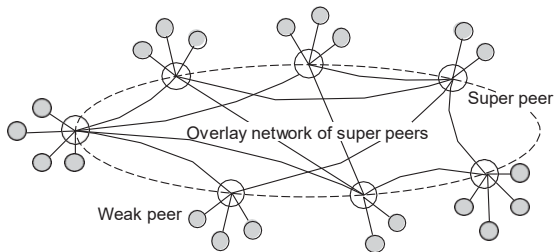
- ▶ 如果 $r/N = 0.001$ ，那么 $S \approx 1000$
- ▶ 采用泛洪且 $d = 10$ 、 $k = 4$ 时，我们会触及 7290 个节点。
- ▶ 随机漫步的通信效率更高，但找到结果所需的时间可能更长。

超节点网络

本质

有时打破纯对等网络的对称性是明智的：

- ▶ 在无结构P2P系统中进行搜索时，使用索引服务器可以提高性能
- ▶ 决定数据存储位置通常可以通过代理更有效地完成



边缘服务器架构Edge-server architecture

本质

部署在互联网上的系统，其服务器位于网络**边缘**：即企业网络与实际互联网之间的边界。

